

WEEK-8

TASK 1 - Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
ramya@LAPTOP-OPDAF309: ~$ nano task1.c
GNU nano 8.7.1 task1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    int *p;
    int *q;
    int i;

    printf("TASK 1: Memory Allocation Demonstration\n");
    printf("-----\n");

    printf("\n1. Allocating memory using malloc()...\n");
    p = (int *)malloc(5 * sizeof(int));

    if (p == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
    {
        p[i] = (i + 1) * 10;
    }

    printf("malloc allocated 5 integers.\n");
    printf("Values: ");

    for (i = 0; i < 5; i++)
    {
        printf("%d ", p[i]);
    }

    printf("\n");
    sleep(5);
}
```

```
printf("\n2. Allocating memory using calloc()...\n");
q = (int *)calloc(5, sizeof(int));

if (q == NULL)
{
    printf("calloc failed\n");
    free(p);
    return 1;
}

printf("calloc allocated 5 integers.\n");
printf("Initial calloc values: ");

for (i = 0; i < 5; i++)
{
    printf("%d ", q[i]);
}

printf("\n");
sleep(5);

printf("\n3. Increasing malloc memory using realloc()...\n");
p = (int *)realloc(p, 10 * sizeof(int));

if (p == NULL)
{
    printf("realloc failed\n");
    free(q);
    return 1;
}

for (i = 5; i < 10; i++)
{
```

```
        p[i] = (i + 1) * 10;
    }

    printf("Memory increased from 5 integers to 10 integers.\n");
    printf("New values: ");

    for (i = 0; i < 10; i++)
    {
        printf("%d ", p[i]);
    }

    printf("\n");
    sleep(5);

    printf("\n4. Memory is currently allocated.\n");
    printf("Press Enter to continue to free()...\n");
    getchar();

    printf("\n5. Releasing memory using free()...\n");

    free(p);
    free(q);

    printf("malloc/calloc/realloc memory has been released.\n");

    printf("\nProgram will remain here for 10 seconds.\n");
    printf("This gives you time to observe the program.\n");

    sleep(10);

    printf("\nProgram completed successfully.\n");

    return 0;
}
```

ramya@LAPTOP-OPDAF309: ~

```
ramya@LAPTOP-OPDAF309:~$ mkdir Prac7
ramya@LAPTOP-OPDAF309:~$ cd Prac7
ramya@LAPTOP-OPDAF309:~/Prac7$ nano task1.c
ramya@LAPTOP-OPDAF309:~/Prac7$ gcc -g task1.c -o task1
ramya@LAPTOP-OPDAF309:~/Prac7$ ./task1
TASK 1: Memory Allocation Demonstration
-----
```

1. Allocating memory using malloc()...
malloc allocated 5 integers.
Values: 10 20 30 40 50

2. Allocating memory using calloc()...
calloc allocated 5 integers.
Initial calloc values: 0 0 0 0 0

3. Increasing malloc memory using realloc()...
Memory increased from 5 integers to 10 integers.
New values: 10 20 30 40 50 60 70 80 90 100

4. Memory is currently allocated.
Press Enter to continue to free()...

5. Releasing memory using free()...
malloc/calloc/realloc memory has been released.

Program will remain here for 10 seconds.
This gives you time to observe the program.

```

ramya@LAPTOP-OPDAF309:~/Prac7$ valgrind --leak-check=full --track-origins=yes ./task1
==697== Memcheck, a memory error detector
==697== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==697== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==697== Command: ./task1
==697==
TASK 1: Memory Allocation Demonstration
-----

1. Allocating memory using malloc()...
malloc allocated 5 integers.
Values: 10 20 30 40 50

2. Allocating memory using calloc()...
calloc allocated 5 integers.
Initial calloc values: 0 0 0 0 0

3. Increasing malloc memory using realloc()...
Memory increased from 5 integers to 10 integers.
New values: 10 20 30 40 50 60 70 80 90 100
^[[C
4. Memory is currently allocated.
Press Enter to continue to free()...

5. Releasing memory using free()...
malloc/calloc/realloc memory has been released.

```

```

Program will remain here for 10 seconds.
This gives you time to observe the program.

```

```

Program completed successfully.
==697==
==697== HEAP SUMMARY:
==697==   in use at exit: 0 bytes in 0 blocks
==697==   total heap usage: 5 allocs, 5 frees, 2,128 bytes allocated
==697==
==697== All heap blocks were freed -- no leaks are possible
==697==
==697== For lists of detected and suppressed errors, rerun with: -s
==697== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

TASK 2 - Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process. give the commands for this and i need everything in one terminal only and dont evry less time in the code as it is running very fast.

```
ramya@LAPTOP-OPDAF309: ~$ nano task2.c
GNU nano 8.7.1 task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE 100000000

int main()
{
    int *data;
    pid_t pid;

    printf("TASK 2: Copy-on-Write Demonstration\n");
    printf("-----\n");

    printf("\nParent: Allocating %d integers...\n", SIZE);
    data = (int *)malloc(SIZE * sizeof(int));

    if (data == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("Parent: Memory allocated successfully.\n");
    printf("\nParent: Initializing memory...\n");

    for (int i = 0; i < SIZE; i++)
    {
        data[i] = i;
    }

    printf("Parent: Memory initialization completed.\n");
    printf("\nParent PID = %d\n", getpid());
```

```
printf("\nParent PID = %d\n", getpid());

printf("\nParent is now waiting for 10 seconds.\n");
printf("You can observe the process memory using another terminal if ne>

sleep(10);

printf("\nCreating child process using fork()...\n");

pid = fork();

if (pid < 0)
{
    printf("fork failed\n");
    free(data);
    return 1;
}

if (pid == 0)
{
    printf("\n===== CHILD PROCESS =====\n");
    printf("Child PID = %d\n", getpid());
    printf("Parent PID = %d\n", getppid());

    printf("\nChild initially shares the memory pages with parent.\n");
    printf("This is the Copy-on-Write stage.\n");

    sleep(10);

    printf("\nChild is now modifying the memory...\n");

    for (int i = 0; i < SIZE; i++)
    {
        data[i] = data[i] + 100;
    }
}
```

```

    printf("Child has modified the memory.\n");
    printf("Because the child modified the pages, Copy-on-Write occurs.\n");

    printf("\nChild will now wait for 15 seconds.\n");
    printf("Observe the child process memory during this time.\n");

    sleep(15);

    printf("\nChild: data[0] = %d\n", data[0]);
    printf("Child: data[99999999] = %d\n", data[99999999]);

    printf("\nChild is freeing its memory...\n");
    free(data);

    printf("Child completed.\n");

    exit(0);
}
else
{
    printf("\n===== PARENT PROCESS =====\n");
    printf("Parent PID = %d\n", getpid());
    printf("Child PID = %d\n", pid);

    printf("\nParent and child now have separate processes.\n");
    printf("Initially, their memory pages are shared using Copy-on-Write\n");

    sleep(5);

    printf("\nParent is NOT modifying the data.\n");
    printf("The parent will wait while the child modifies its memory.\n");

    sleep(20);

    printf("\nParent: data[0] = %d\n", data[0]);
    printf("Parent: data[99999999] = %d\n", data[99999999]);

```

```

    printf("\nParent waiting for child to finish...\n");

    wait(NULL);

    printf("Child process has finished.\n");

    free(data);

    printf("\nParent memory has been freed.\n");
    printf("Copy-on-Write demonstration completed.\n");
}

return 0;
}

```

```
ramya@LAPTOP-OPDAF309:~/Prac7$ nano task2.c
ramya@LAPTOP-OPDAF309:~/Prac7$ gcc -g task2.c -o task2
ramya@LAPTOP-OPDAF309:~/Prac7$ ./task2
TASK 2: Copy-on-Write Demonstration
-----

Parent: Allocating 10000000 integers...
Parent: Memory allocated successfully.

Parent: Initializing memory...
Parent: Memory initialization completed.

Parent PID = 814

Parent is now waiting for 10 seconds.
You can observe the process memory using another terminal if needed.
```

```
===== PARENT PROCESS =====
Parent PID = 814
Child PID = 815

Parent and child now have separate processes.
Initially, their memory pages are shared using Copy-on-Write.

===== CHILD PROCESS =====
Child PID = 815
Parent PID = 814

Child initially shares the memory pages with parent.
This is the Copy-on-Write stage.

Parent is NOT modifying the data.
The parent will wait while the child modifies its memory.

Child is now modifying the memory...
Child has modified the memory.
Because the child modified the pages, Copy-on-Write occurs.

Child will now wait for 15 seconds.
Observe the child process memory during this time.

Parent: data[0] = 0
Parent: data[99999999] = 9999999

Parent waiting for child to finish...

Child: data[0] = 100
Child: data[99999999] = 10000099

Child is freeing its memory...
Child completed.
Child process has finished.

Parent memory has been freed.
Copy-on-Write demonstration completed.
```